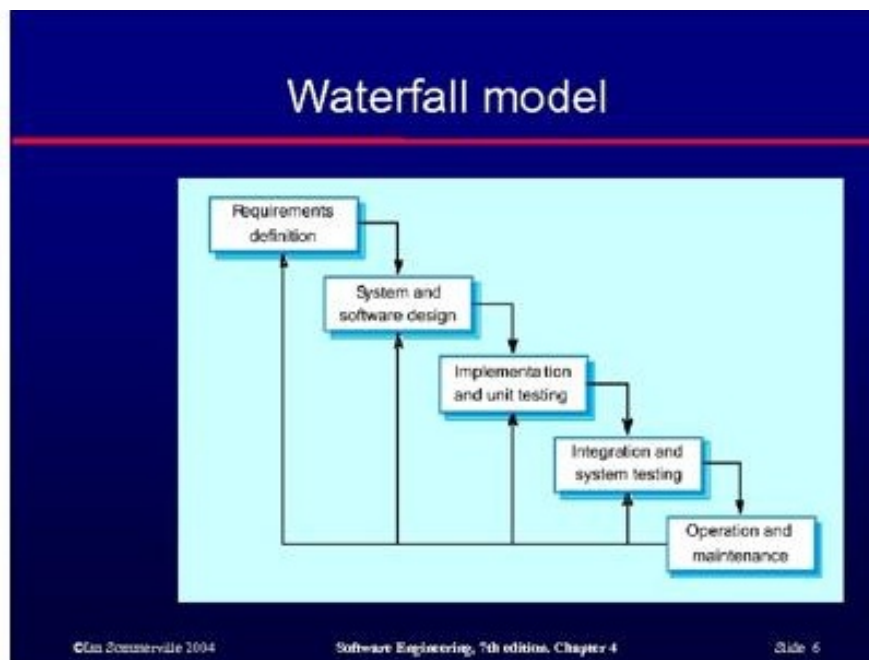


Software Development Life Cycle Models

Waterfall • Incremental • Prototyping • Spiral • Agile

A classroom-oriented Software Engineering note for undergraduate CSE students



Incremental model

The aim of this note is not to make students memorize five definitions. The idea is to understand how each model changes the way a software team plans, develops, tests, delivers, and responds to change.

1. Understanding SDLC Before Studying the Models

When a software project begins, the team does not immediately start writing code. There is a problem to understand, people to consult, requirements to clarify, a design to prepare, software to build, and a system to test and maintain. The **Software Development Life Cycle (SDLC)** is the organized way in which these activities are carried out.

Different SDLC models do not necessarily contain completely different activities. The important difference is **how those activities are arranged**. In a Waterfall approach, the work moves mainly in a sequence. In an Incremental approach, a large product is divided into usable pieces. Prototyping introduces an early model of the proposed system so that users can react to it. Spiral development repeatedly examines risk, while Agile organizes development into short cycles with frequent feedback. These distinctions are central to understanding software engineering.

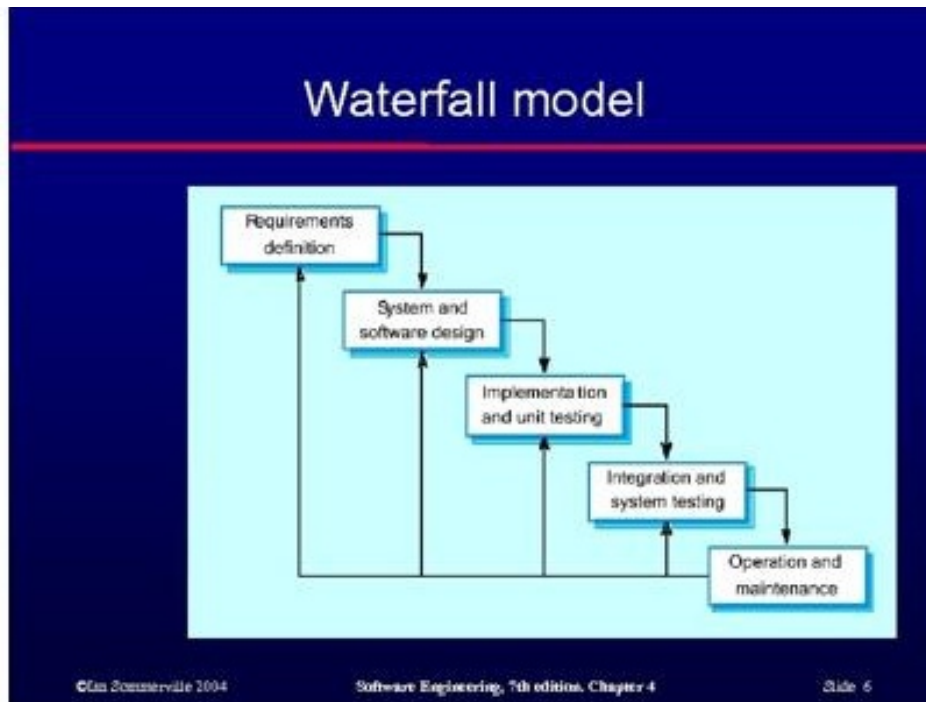
A useful way to think about SDLC models is to ask five questions. How certain are the requirements? How expensive would a mistake be? How often will users need to give feedback? Does the customer need a working product early? And how much change is expected during development? The answers usually point toward a suitable process model.

The five models covered in this class are widely discussed in software engineering literature. OpenStax, for example, describes Waterfall, Incremental, Prototyping, Spiral and Agile process approaches and emphasizes that there is no single process that is appropriate for every project. ■cite■turn0search0■

A small but important distinction

An SDLC model is not the same thing as a programming language, development tool, or project-management software. A team can use Java, Python, Git, cloud services, automated testing, and the same database technology under very different SDLC models. The model describes the *process of development*, not the programming technology.

2. Waterfall Model



Incremental model

Figure: Waterfall process diagram supplied for classroom use. The supplied diagram retains its original source credit.

The basic idea

The Waterfall model is the classic sequential software process model. The project begins with requirements, moves to system and software design, continues through implementation and testing, and eventually reaches operation and maintenance. The name comes from the way the phases are traditionally drawn: one phase flows into the next, much like water moving downward.

In a strict interpretation, the team tries to complete and approve one major phase before moving to the next. This gives management a clear sequence of deliverables. Requirements documentation is prepared first; designers use those requirements to produce the design; programmers implement the design; testers evaluate the completed software; and the released system enters operation and maintenance.

What happens in practice?

Imagine that a university wants a payroll system. The university already knows how salaries, deductions, allowances, tax calculations, employee categories, approval procedures, and reports should work. The requirements are therefore relatively stable. In such a situation, spending considerable time defining the requirements before implementation can make sense.

The difficulty appears when the assumptions are wrong. Suppose that after six months the payroll department discovers that a major workflow was misunderstood. Changing the requirement may affect the design, database, code, test cases, documentation, and schedule. The later the change appears, the more expensive the rework can become.

Advantages

Waterfall's greatest strength is its clarity. A project manager can see which phase is currently active, what documents should exist, and what approval is needed before the next phase begins. It is therefore useful where requirements are well understood and where formal documentation and predictable milestones matter.

Another strength is that the model encourages deliberate planning. Teams can estimate resources and prepare a substantial design before implementation starts. This can be valuable in environments where traceability, contracts, or formal approvals are important.

Disadvantages

The major weakness is limited adaptability. Real users often discover new needs only after they see a working system. Waterfall assumes that requirements can be understood sufficiently early to support the later stages. When that assumption fails, changes become costly.

Testing also tends to produce important feedback relatively late compared with iterative approaches. A team may discover a serious misunderstanding only after much of the system has already been implemented.

When should you choose Waterfall?

Choose it when the requirements are stable, the scope is well defined, the technology is reasonably understood, and the project benefits from formal phase-by-phase control. Do not choose it merely because it is easy to draw. The nature of the project should justify the model.

Real-life analogy: Building a house from an approved architectural plan is a useful analogy. Major structural changes after construction begins are expensive. Software is not exactly the same as construction, but the analogy helps students understand why early decisions matter in a sequential model.

3. Incremental Model

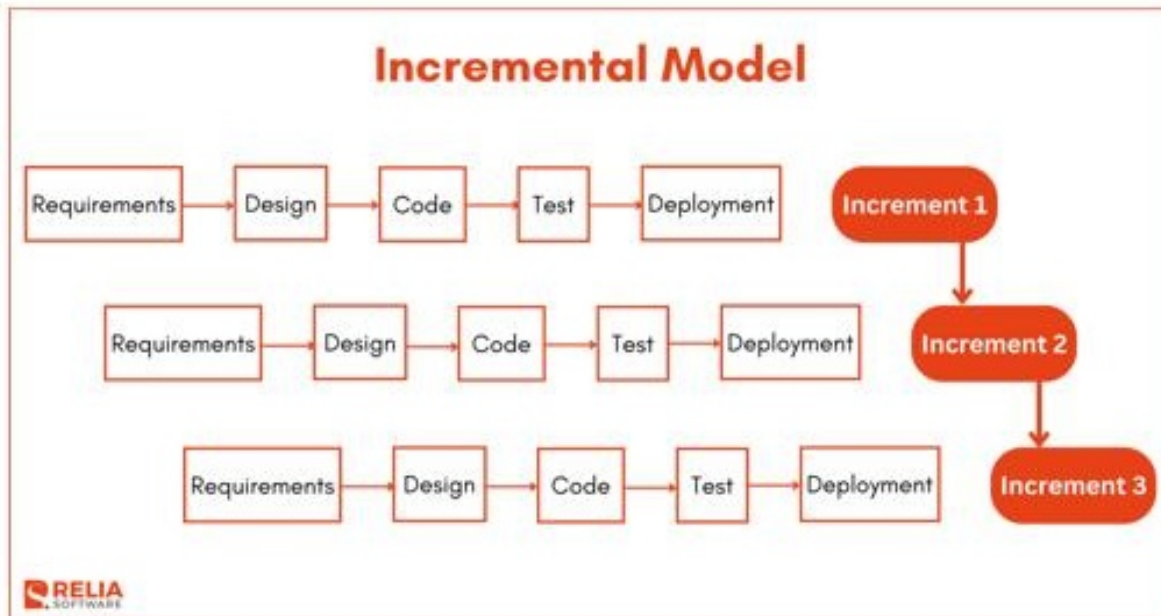


Figure: Incremental Model diagram supplied for classroom use. The diagram shows repeated requirement–design–coding–testing–deployment cycles producing successive increments.

The basic idea

The Incremental model starts with an overall understanding of the product but does not wait until every feature has been delivered before giving users something useful. The system is divided into increments, and each increment goes through the necessary development and testing activities.

Think of a university e-learning platform. The first increment might provide student login, course browsing, and lecture access. A second increment could introduce quizzes and assignment submission. A later increment could add discussion forums, analytics, and notifications. Each increment is part of the same product, but users do not have to wait for the entire system before receiving useful functionality.

The important word here is **increment**. An increment is not simply a piece of unfinished code. Ideally, it is a meaningful addition to the system that can be integrated, tested, and delivered. This is why incremental development can provide visible progress and useful feedback.

Why is it different from Waterfall?

Waterfall normally treats the product as one large delivery after the major phases have been completed. Incremental development treats delivery as a sequence of smaller product additions. The overall architecture and requirements may still be planned, but implementation and delivery are distributed across increments.

Advantages

The most obvious advantage is early delivery. Users can start using important functions while lower-priority functions are still under development. This can be particularly valuable when the organization has a pressing business need and cannot wait for the complete system.

Testing also becomes more manageable because each increment is smaller. Feedback from an early increment can reveal usability or functional problems before the entire system has been built. Progress is easier to demonstrate because the team can point to completed increments rather than only to documents.

Disadvantages

Incremental development requires good architectural thinking. If the team builds each increment independently without considering the whole product, integration problems can accumulate. A quick solution for one increment may later make another feature difficult to implement.

Prioritization is another challenge. The team must decide which functionality should come first. If the wrong features are prioritized, users may receive several releases without receiving the capability that actually matters most.

When should you choose it?

The model is a good candidate when the product can be divided into coherent functional groups and early delivery has business value. Banking portals, e-learning platforms, administrative systems, and mobile applications can often be organized this way.

4. Prototyping Model

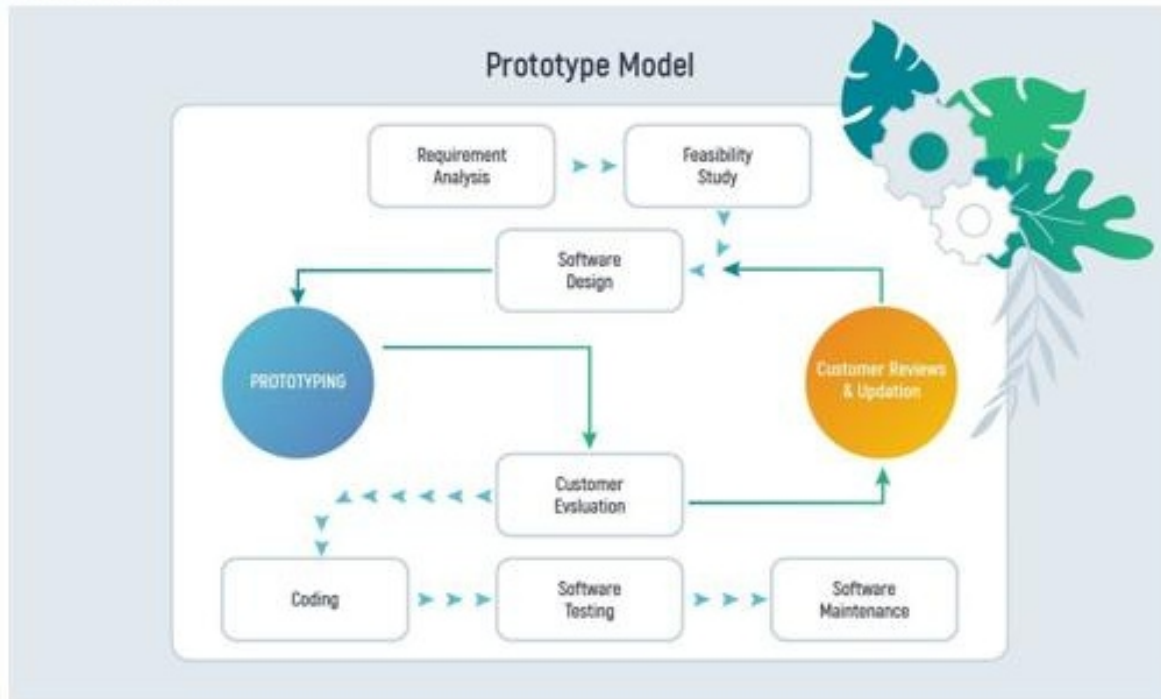


Figure: Prototyping Model diagram supplied for classroom use. The diagram illustrates requirements, quick development, customer evaluation and refinement before the final product.

Why do we need a prototype?

A common problem in software engineering is that users may know what they want but may not be able to describe it precisely. This is especially common with screens, workflows, dashboards, mobile applications, and new systems for which there is no familiar example.

A prototype gives the conversation something concrete. Instead of asking a doctor, 'What should the dashboard look like?', the development team can show a sample dashboard. The doctor may immediately notice that the patient information should appear on the left, that an alert should be more prominent, or that two screens should actually be combined.

The prototype may be a clickable interface, a mock-up, a limited working program, or another representation of the proposed solution. Its purpose is learning and validation. It is not automatically the final production system.

A typical cycle

The team first gathers an initial set of requirements and creates a quick design. A prototype is then produced and shown to users. Users review it and identify missing or incorrect features. The requirements and prototype are refined, and the cycle continues until the team has sufficient confidence to design and build the actual system.

Advantages

The strongest benefit is improved communication. A prototype reduces the gap between what a stakeholder says and what the development team thinks that statement means. It can also expose usability problems early, when they are relatively inexpensive to correct.

Prototyping is particularly valuable for user-interface design. A screen that looks reasonable to a developer may be confusing to an actual user. Early user evaluation can reveal these problems before the team invests heavily in the final implementation.

Disadvantages

There is an important danger: users may see a prototype that looks polished and assume that the underlying system is almost complete. In reality, a prototype may have little security, poor performance, temporary data handling, incomplete error handling, or no production-grade architecture.

For this reason, the team must clearly explain what the prototype is intended to demonstrate. It is also possible for repeated feedback cycles to consume significant time. OpenStax similarly notes that prototype evaluation can become time-consuming and that a prototype may appear functional while still lacking the internal functionality required for the final system. ■cite■turn0search0■

A practical example

Suppose a hospital is developing a doctor dashboard. Doctors may disagree about the ideal workflow. Building the complete system first would be risky. A prototype allows the hospital to test the workflow with real users, learn from their reactions, and then invest in the final architecture.

5. Spiral Model

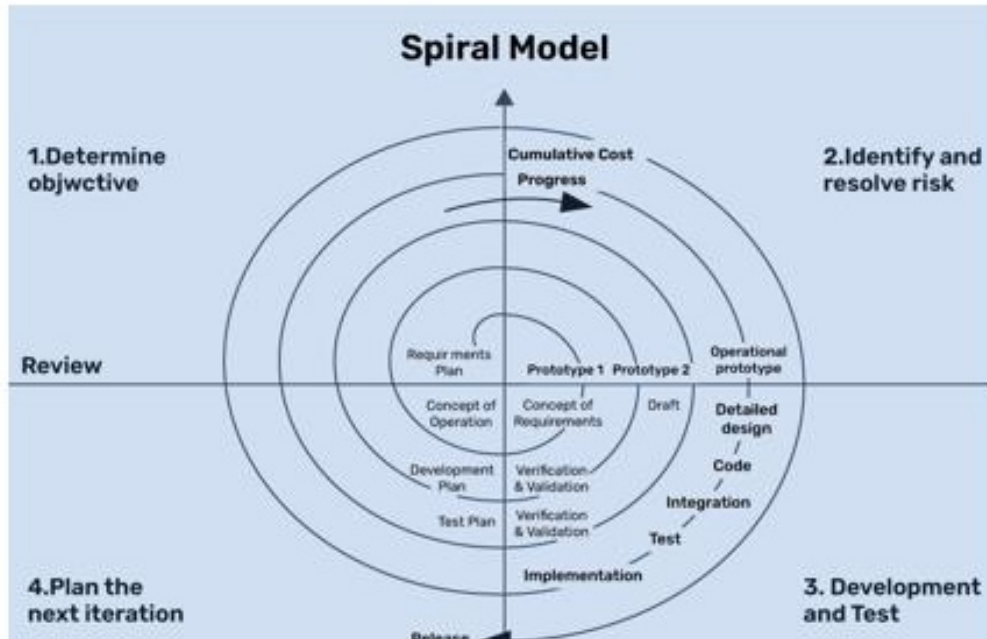


Figure: Spiral Model diagram supplied for classroom use. The spiral illustrates repeated cycles involving objectives, risk analysis, development/testing, and planning for the next iteration.

The central idea

The Spiral model is designed around a simple observation: some software projects are dangerous to manage because the team does not yet know which risks will cause the most trouble. Instead of pretending that everything is known at the beginning, the Spiral model repeatedly identifies objectives, examines alternatives and risks, develops and evaluates a solution, and then plans the next cycle.

Risk is therefore not an activity that happens only once. It is a continuing concern. The team might identify security risk in one cycle, scalability risk in another, and integration risk in a later cycle. Each spiral loop is an opportunity to learn and reduce uncertainty.

Consider a large digital identity platform. Before building the complete system, the team may need to investigate biometric accuracy, identity verification, privacy, database scalability, interoperability with government systems, and cybersecurity. A risk-driven approach is attractive because the consequences of a wrong decision could be enormous.

How the spiral is read

The diagram is usually shown as a spiral because each loop represents another round of development. Early loops may concentrate on feasibility and high-risk questions. Later loops move toward a more complete operational system. The spiral does not simply mean 'keep developing repeatedly'; the repeated cycles are guided by explicit risk evaluation.

Advantages

The main strength is risk management. A team can investigate the most uncertain or dangerous part of a project before committing to a complete implementation. This makes the model particularly attractive for large, complex, expensive projects.

Another benefit is that the model naturally accommodates learning. The results of one cycle influence the objectives and decisions of the next. Stakeholders can also review progress at meaningful points.

Disadvantages

The model is more demanding to manage than a simple sequential process. Effective risk analysis requires experienced people who can identify, evaluate, and prioritize technical and business risks. If the project team does not have that capability, the model can become unnecessarily complicated.

It can also be expensive for small projects. There is little value in performing elaborate risk analysis for a simple student calculator application. The process should be proportional to the consequences and uncertainty of the project.

Key exam point: If a question emphasizes *high risk, uncertainty, large scale, expensive failure, or repeated risk analysis*, Spiral should immediately come to mind. Standard software-engineering references describe Spiral as combining iterative development with a strong focus on reducing project risk. ■cite■turn0search2■

6. Agile Model

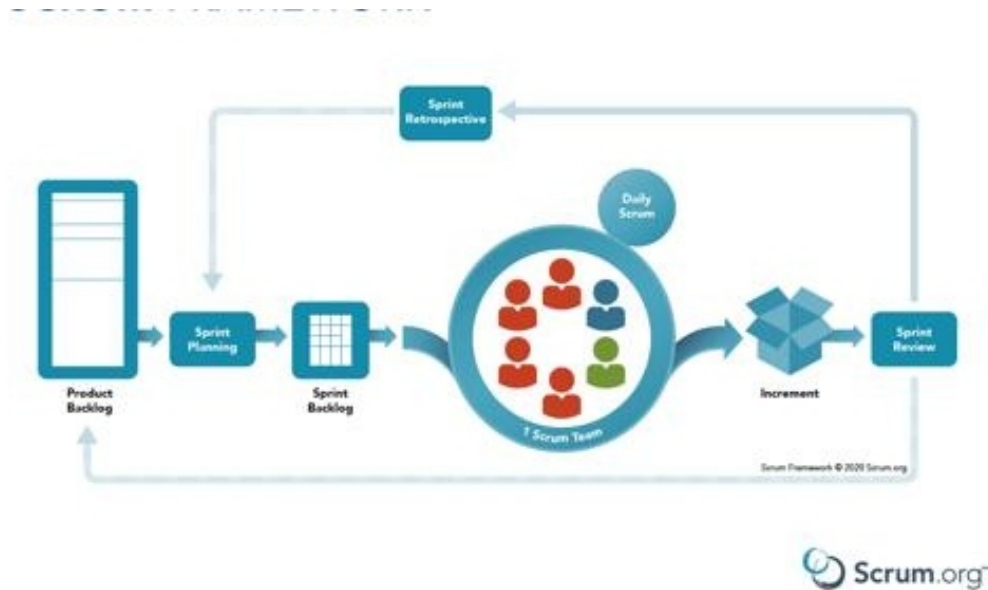


Figure: Scrum framework diagram. Scrum is an Agile framework; Agile itself is the broader approach to adaptive software development.

Agile is more than a diagram

Agile development became important because software projects often operate in environments where requirements can change quickly. A customer may learn something new after seeing the first version of the product. A competitor may introduce a new feature, a regulation may change, or a business idea may prove less useful than expected. Agile treats this change as something the development process must be able to handle.

Instead of planning every detail months in advance and treating the plan as fixed, Agile teams work in short cycles. They select a manageable set of high-priority work, develop and test it, show the result to stakeholders, learn from the feedback, and then decide what should happen next.

Understanding the Scrum diagram

The diagram shows a Product Backlog leading into Sprint Planning. A selected set of work becomes the Sprint Backlog. During the sprint, the Scrum Team works toward an Increment. Daily Scrum meetings help the team coordinate, while Sprint Review and Sprint Retrospective provide opportunities to inspect the product and improve the way the team works. The important idea is the repeated loop: plan, build, inspect, learn, and adapt.

Advantages

Agile is particularly strong when the customer does not know every requirement at the beginning. Because working software is produced frequently, stakeholders can respond to something real rather than only reading a long requirements document.

Another advantage is prioritization. A team can focus first on the features that deliver the greatest value. If priorities change, the next cycle can reflect those changes. This reduces the risk of spending months developing a feature that users ultimately do not need.

6. Agile Model — Continued

Disadvantages

Agile does not mean that planning, architecture, testing, or documentation are unnecessary. A poorly implemented Agile process can become chaotic. Teams still need technical discipline, quality standards, version control, testing, security practices, and a clear product direction.

Predicting the exact final scope and date can also be harder when requirements are deliberately allowed to change. In addition, large or safety-critical systems may require more formal governance and documentation than a lightweight Agile process provides.

A real-life example

Consider a food-delivery application. The business may initially want restaurant search and ordering. After release, customers may request live delivery tracking, different payment options, better offers, or a simpler checkout process. An Agile team can use feedback from each release to decide which improvement should be developed next.

Classroom observation

Agile should not be confused with 'no planning'. Agile teams plan continuously. The difference is that planning is revisited as new information becomes available. This makes Agile particularly useful when learning and change are expected parts of the project.

Remember: Waterfall asks the team to plan a large sequence in advance; Agile accepts that the team may learn important information during development and therefore keeps the next decisions flexible.

7. Comparing the Five Models

The following comparison is more useful than memorizing five isolated definitions. When you are given a problem in an examination or classroom discussion, identify the dominant characteristic of the project and then select the model that responds to that characteristic.

Model	Central idea	Requirement flexibility	Main strength	Main concern
Waterfall	Sequential phases	Low	Planning and control	Late change is costly
Incremental	Deliver in functional pieces	Medium–High	Early useful delivery	Integration and prioritization
Prototyping	Learn through an early model	High	Requirement clarification	Prototype may create false expectations
Spiral	Iterate around risk	High	Risk reduction	Cost and management complexity
Agile	Short cycles and feedback	Very high	Adaptability and frequent delivery	Requires discipline and collaboration

A useful mental map

Stable requirements → **Waterfall**. If the requirements are clear and unlikely to change, sequential planning may be appropriate.

Need usable features in stages → **Incremental**. If the product can be divided into meaningful feature groups, deliver those groups progressively.

Users cannot clearly explain what they want → **Prototyping**. Show them something concrete, learn from their feedback, and refine the understanding.

Project risk is the dominant problem → **Spiral**. Identify and attack the most serious risks through successive cycles.

Requirements and priorities change frequently → **Agile**. Work in short cycles, deliver value, collect feedback, and adapt.

8. Real-Life Case Studies for Classroom Discussion

Case 1 — University Payroll System

The university already has documented salary rules, deduction formulas, employee categories, approval procedures, and reporting formats. Management wants formal approval at major milestones and expects the scope to remain stable. **Waterfall** is a reasonable starting choice because the requirements are comparatively predictable and documentation matters.

Case 2 — E-Learning Platform

The university wants students to access courses quickly. Login and course viewing are essential, while quizzes, assignment submission, discussion forums, analytics, and notifications can be added later. **Incremental development** fits naturally because each release can add a useful group of features.

Case 3 — Hospital Doctor Dashboard

Doctors understand their work but are uncertain about the ideal screen layout. A development team that immediately builds the complete application may spend months implementing a workflow that doctors dislike. A **prototype** allows the team to test the interaction before making a large investment.

Case 4 — National Digital Identity System

The project involves security, privacy, scalability, interoperability, and very high consequences if major technical decisions are wrong. The team should investigate the highest risks early and repeatedly. A **Spiral**-style risk-driven process is a strong candidate.

Case 5 — Food-Delivery Application

The business environment changes rapidly. Customer feedback, competitor behavior, promotions, payment methods, and delivery expectations can change priorities. An **Agile** approach allows the product backlog to evolve and working increments to be delivered frequently.

Notice the pattern

The correct answer is not obtained by asking which model is 'modern' or which model is 'best'. The answer comes from matching the process to the project. A stable payroll system and a rapidly changing mobile application have different process needs, even though both are software products.

9. Questions and Classwork

The questions below are intentionally mixed. Some test definitions, while others require students to reason about a real project. For the scenario questions, students should explain *why* they selected a model rather than giving only its name.

1. What is SDLC? Explain why an organization needs a software process model.
2. Explain the Waterfall model in your own words. Why is it called Waterfall?
3. A client changes a major requirement after coding has already progressed substantially. Why can this be difficult in Waterfall?
4. A university wants a payroll system with stable requirements and formal documentation. Which model would you recommend? Give two reasons.
5. Differentiate between Waterfall and Incremental development.
6. Why is an increment expected to provide useful functionality rather than merely unfinished code?
7. An e-learning system must release login and course access first and analytics later. Which model fits this situation? Explain.
8. What is a prototype? Why can a prototype improve communication between users and developers?
9. A hospital doctor says, 'I cannot explain the screen I need, but I will know it when I see it.' Which model or technique would help most?
10. Why can users misunderstand the status of a prototype?
11. Explain the main idea of the Spiral model.
12. Why is risk analysis especially important in the Spiral model?
13. Give an example of a project where Spiral would be unnecessarily expensive.
14. What does Agile mean in software development?
15. Explain the relationship among Product Backlog, Sprint, Increment, Sprint Review, and Retrospective in Scrum.
16. Why is Agile useful when requirements change frequently?
17. Does Agile mean that documentation and planning are unnecessary? Explain.
18. A company wants to release a food-delivery application every few weeks and use customer feedback to decide the next features. Which model would you recommend?
19. Compare Waterfall and Agile with respect to requirements, customer feedback, delivery, and change.
20. Compare Incremental and Agile. Explain why the two ideas are related but not identical.
21. A project has unclear requirements but low technical risk. Would you first consider Prototyping or Spiral? Justify your answer.
22. A project has stable requirements but very high technical and business risk. Which factor should influence your choice most strongly?
23. Can an organization combine Waterfall-style documentation with Agile development cycles? Discuss.
24. Which model would you select for a small college library application? State your assumptions before answering.
25. Which model would you select for a rapidly evolving mobile application? Give a practical justification.
26. Which model is most strongly associated with continuous risk analysis?
27. Which model is most strongly associated with delivering functionality in separate increments?

28. Which model is most useful when the user needs to see an early representation of the proposed system?
29. Write one situation where Waterfall would be a better choice than Agile.
30. Write one situation where Agile would be a better choice than Waterfall.

10. Final Revision: Remember the Reason, Not Just the Name

If you remember only one idea from this class, remember this: **an SDLC model is a way of organizing development decisions**. Waterfall emphasizes a planned sequence. Incremental development emphasizes delivering the product in functional pieces. Prototyping emphasizes learning what the user actually needs. Spiral emphasizes reducing risk through repeated analysis and development. Agile emphasizes adaptation, collaboration, short cycles, and frequent delivery.

In a good software engineering answer, do not stop after writing the model's name. State the project condition, explain how the model responds to that condition, and then mention one advantage and one limitation. That small structure turns a memorized definition into a proper engineering justification.

Reference note

The conceptual explanations in this note were checked against standard software-engineering teaching material, including OpenStax's *Introduction to Computer Science*, Section 9.2, which discusses traditional and Agile software process models. ■cite■turn0search0■turn0search1■ The diagrams in this classroom PDF are the diagrams supplied in the conversation for teaching use; their original source credits remain visible where present.